Jak zbudowaliśmy „Obecność" — poradnik od zera

Ten dokument opisuje **każdy krok** potrzebny do zbudowania i wypchnięcia do internetu tej aplikacji: od pustego folderu do `https://mi.zur-i.com`. Przy każdym kroku wyjaśnione jest **po co to robimy**, **co się stanie jeśli tego nie zrobimy** oraz **jakie są alternatywy**.

Zakładam, że znasz podstawy terminala i Gita. Nic więcej.

Spis treści

1. Mapa całości — co, gdzie, dlaczego
2. Narzędzia lokalne — co trzeba zainstalować
3. Struktura repo — monorepo czy osobno?
4. Frontend: Vite + React
5. Backend: Node + Express
6. Baza: Postgres w Dockerze
7. Docker Compose — spinamy backend i bazę
8. Proxy w Vite — parzystość dev/prod
9. React Router i pułapka odświeżania strony
10. Serwer: wynajęcie EC2 + DNS
11. Nginx jako reverse proxy
12. HTTPS z Let's Encrypt
13. CI/CD z GitHub Actions
14. Obserwowalność i utrzymanie
15. Alternatywy globalne — Vercel, Render, Fly.io

---

1. Mapa całości

```
┌──────────────┐
│  Przeglądarka │
└──────┬───────┘
       │ HTTPS
┌──────▼──────────────────────────────┐
│  EC2 (Ubuntu)                       │
│  ┌──────────────┐                   │
│  │  Nginx :443  │                   │
│  │   ├ /        → /var/www/mi/*    │
│  │   └ /api/*   → 127.0.0.1:3001   │
│  └──────┬───────┘                   │
```

```
│          │                                │
│  ┌───────▼──────── docker compose ──┐    │
│  │  backend (Node/Express) :3000    │    │
│  │         │                        │    │
│  │  db (Postgres 16)  :5432 ◄───────┘    │
│  └──────────────────────────────────┘    │
└──────────────────────────────────────────┘

            ▲
            │ git push origin main
┌───────────┴──────────┐
│ GitHub + Actions     │
│ (deploy po SSH)      │
└──────────────────────┘
```

Dlaczego tak?

- Frontend i backend to osobne światy — frontend to „zbudowane raz, potem statyczne pliki"; backend to „proces, który musi chodzić cały czas". Nie opłaca się ich mieszać.
- Nginx z przodu, bo: (a) serwuje statyki wydajnie, (b) robi HTTPS, (c) ukrywa Node, (d) może łatwo dołożyć rate-limiting, cache, kompresję.
- Docker Compose, bo backend + baza to zawsze para. Compose to jeden plik, który opisuje obie rzeczy, ich zależność (backend czeka aż db jest zdrowa), i jak się łączą przez prywatną sieć.

---

2. Narzędzia lokalne

Na laptopie potrzebujesz:

Narzędzie	Po co	Co jeśli nie zainstalujesz
Git	wersjonowanie, push na GitHub	nie ma deployu
Node.js 20 + npm	frontend i backend	nic lokalnie się nie odpali
Docker Desktop	Postgres lokalnie, test Compose	musisz instalować Postgres natywnie (co jest zawsze ból w tyłku)
GitHub CLI (`gh`)	wygodny push/PR/actions	musisz klikać w UI — da się żyć
Edytor (VS Code, Cursor)	edycja kodu	notatnik wystarczy, ale po co

Alternatywa dla Node.js: Bun albo Deno. Oba szybsze, ale ekosystem Express

- `pg` jest najbardziej wygrzany pod Node — zostajemy przy Node.

Alternatywa dla Dockera: natywny Postgres (`brew install postgresql@16`). Problem: trzeba ręcznie uruchamiać, każdy członek zespołu ma inną wersję, produkcja jest w kontenerze a dev nie — **dryft środowisk** gwarantowany.

---

3. Struktura repo

Wybraliśmy **monorepo**: frontend i backend w jednym repozytorium, w podkatalogach.

```
vc/
├── README.md
├── docker-compose.yml
├── nginx.conf
├── .github/workflows/deploy.yml
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
└── frontend/
    ├── vite.config.js
    ├── package.json
    └── src/
```

Dlaczego monorepo?

- **Atomowe zmiany** — jednym commitem zmieniamy schema bazy (backend) i formularz który z niej korzysta (frontend). Gdyby były w osobnych repo, musiałbyś robić koordynowane merge'e dwóch PR-ów.
- **Jeden deploy** — nasz CI/CD ściąga jedno repo, buduje obie części.
- Mały projekt = mało złożoności = monorepo wystarczy.

Kiedy osobne repa mają sens?

- Duży zespół, różne cykle wydań.
- Backend wdrażany kilka razy dziennie, frontend raz w tygodniu.
- Różne poziomy uprawnień (np. frontend open-source, backend zamknięty).

Krok 1:

```
mkdir vc && cd vc
git init
gh repo create mi-zuri/vc --private --source=. --remote=origin
```

`★ Insight ─────────────────────────────────`

- `gh repo create` w jednym kroku tworzy repo na GitHubie, ustawia remote i jest autoryzowane przez GitHub CLI — dużo szybciej niż przez UI.
- `--private` na start jest zdrową domyślną — łatwiej zrobić publiczne potem niż cofnąć przypadkowe wyciekanie kluczy z commit history.

---

4. Frontend: Vite + React

4.1 Po co Vite?

Vite to **narzędzie developerskie i build tool** dla frontendów:

- W trybie `dev` — błyskawiczny serwer z HMR (Hot Module Replacement), odpala się w ~200 ms.
- W trybie `build` — pakuje wszystko (JS, CSS, obrazki) w zminifikowane paczki do folderu `dist/` — statyczne pliki gotowe do wrzucenia na CDN/Nginx.

Alternatywy:

- **Create React App (CRA)** — przestarzały, porzucony przez Facebooka. Nie.
- **Next.js** — framework, nie tylko build tool. Rozwiązuje problemy których nie mamy (SSR, routing serwerowy). Dla naszej skali to przerost formy nad treścią.
- **esbuild/webpack raw** — sam konfigurujesz wszystko. Możesz, ale po co.

Co jeśli nie użyjemy żadnego bundlera? Musiałbyś serwować surowe pliki JS z `type="module"`. Działa na małą skalę, ale: brak minifikacji, brak tree-shakingu (nieużywany kod w paczce), brak TypeScriptu bez kombinowania, brak obsługi CSS-ów jako importów.

4.2 Inicjalizacja

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install react-router-dom
```

`--template react` — szablon bez TypeScripta. Nasze wybór dla prostoty; TypeScript byłby lepszy długoterminowo.

Co powstaje:

```
frontend/
├── index.html          # punkt wejścia
├── package.json
```

```
├── vite.config.js
└── src/
    ├── main.jsx        # bootstrap: ReactDOM.render
    ├── App.jsx
    └── index.css
```

4.3 Dlaczego `index.html` jest w korzeniu, nie w `src/`?

Vite traktuje `index.html` jako **główny entry point** — szuka w nim `<script type="module" src="/src/main.jsx">` i od tego zaczyna graf zależności. Gdyby był w `src/`, trzeba by zmieniać domyślne ścieżki. Zostaw jak jest.

4.4 Design — CSS custom properties, Lora, paper-grain

Cały design system to `src/index.css` z tokenami jak `--primary`, `--cream`, i typografią serif (Lora) + sans (Source Sans 3) z Google Fonts.

Alternatywa: Tailwind. Jest szybszy dla prototypów, ale dla prawdziwego „editorial" designu (grube litery, duże marginesy, serif headings) — zwykły CSS z tokenami daje pełną kontrolę i żadnych `className="flex gap-4 md:gap-6 lg:gap-8"`.

`★ Insight ─────────────────────────────────`

- `color-mix(in srgb, var(--primary) 60%, var(--primary-tint))` to funkcja CSS-a która miesza kolory — cały gradient mapy samotności robimy tym, bez generowania 10 kolorów HEX ręcznie.
- Google Fonts wstrzykujemy w `index.html` (a nie jako `@import` w CSS-ie) bo `<link rel="preconnect">` przyspiesza pierwszy load o 100-300 ms.

`─────────────────────────────────────────────`

---

5. Backend: Node + Express

5.1 Po co Express?

Express to **minimalny framework HTTP** dla Node. Daje ci routing i middleware — nic więcej. „Nic więcej" to zaleta.

Alternatywy:

- **Fastify** — szybszy, nowocześniejszy, ma wbudowaną walidację schem JSON. Dla nowego projektu rozważyłbym go.
- **NestJS** — pełny framework TypeScript w stylu Angulara. Overkill dla API na 10 endpointów.
- **Plain Node `http`** — możesz, ale napiszesz własnego Expressa po drodze.

5.2 Inicjalizacja

```
mkdir backend && cd backend
npm init -y
npm install express pg cors
```

- `pg` — klient Postgresa dla Node.
- `cors` — middleware do nagłówków CORS. W produkcji de facto nieużywany, bo frontend i API są na tej samej domenie, ale na dev się przydaje.

5.3 Struktura

```
backend/src/
├── index.js    # serwer Express + wszystkie endpointy
├── db.js       # pool Postgresa + seed
├── schema.sql  # CREATE TABLE …
└── seeds/      # JSON-owe dane startowe
```

Dlaczego wszystkie endpointy w jednym pliku? Mały projekt. Gdy przekroczy ~300 linii, rozbijamy na `routes/volunteers.js`, `routes/seniors.js` itd. **Premature splitting** to grzech równie duży jak **god file**.

5.4 Pool połączeń

```
import pg from 'pg';
export const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });
```

Po co pool? Każde zapytanie do bazy wymaga połączenia TCP + handshake (~5-20 ms). Pool trzyma kilka otwartych połączeń i je reuseuje — zapytanie trwa milisekundy.

Co jeśli nie użyjesz pool'a? Każdy request robi nowe połączenie — pod obciążeniem zapychasz sobie ograniczenie Postgresa (`max_connections`, domyślnie 100) po kilkudziesięciu równoczesnych requestach.

5.5 Seed przy starcie

W `db.js` przy starcie aplikacji:

1. Czyta `schema.sql` i wykonuje (idempotentne dzięki `CREATE TABLE IF NOT EXISTS`).
2. Patrzy czy `seniors` jest puste — jeśli tak, wczytuje `seeds/seniors.json` i robi `INSERT`-y.

Alternatywa profesjonalna: narzędzia do migracji (**Prisma Migrate**, **Knex**, **Flyway**). Dają historię zmian schema, rollbacki, wersjonowanie. Dla MVP to overkill — ale **pierwszą rzeczą po MVP** jest dołożenie migracji, bo ręczne `ALTER TABLE` na produkcji to przepis na wieczne bóle głowy.

6. Baza: Postgres w Dockerze

6.1 Po co Docker dla bazy?

- **Parzystość dev/prod** — ta sama wersja Postgresa lokalnie i na serwerze.
- **Zero instalacji** — `docker compose up` i masz bazę.
- **Czysta izolacja** — dane w Docker volume, kasujesz jednym poleceniem.

Alternatywa: Postgres jako serwis managed (Amazon RDS, Supabase, Neon). Plusy: backupy, HA, autoscaling, nie musisz się martwić o obsługę. Minusy: koszty ($20+/mc), lock-in, więcej kroków konfiguracyjnych.

Dla MVP wybraliśmy self-hosted w Dockerze — zero kosztów, full kontrola. Przy produkcyjnych danych przeszedłbym na managed.

6.2 `schema.sql`

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE IF NOT EXISTS volunteers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  first_name TEXT NOT NULL,
  ...
);
```

Dlaczego UUID zamiast `SERIAL` (auto-inkrementowane int)?

- **Nie zdradza liczby rekordów** — `?volunteerId=3` pokazuje że masz 3 wolontariuszy; UUID nic nie mówi.
- **Można generować client-side** — offline apps, distributed systems.
- Minus: 16 bajtów zamiast 4-8, mniej wydajne jako klucze w indeksach. Dla naszej skali nieistotne.

Dlaczego `pgcrypto` a nie `uuid-ossp`? `pgcrypto` jest standardowy w Postgresie od lat, `uuid-ossp` wymaga osobnej instalacji rozszerzenia.

---

7. Docker Compose

7.1 Koncept

`docker-compose.yml` opisuje **wiele kontenerów jako jedną aplikację**. Uruchamiasz `docker compose up -d` i wszystko rusza.

Nasz plik:

```yaml
services:
  backend:
    build: { context: ., dockerfile: backend/Dockerfile }
    ports:
      - "127.0.0.1:3001:3000"
    environment:
      DATABASE_URL: postgres://appuser:${DB_PASSWORD}@db:5432/appdb
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: appdb
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 5s
      retries: 5
volumes:
  pgdata:
```

7.2 Rozbiór kluczowych linijek

`"127.0.0.1:3001:3000"` — prefiks `127.0.0.1:` oznacza „nasłuchuj tylko na loopback, nie na publicznym interfejsie sieciowym". Gdyby było samo `3001:3000`, backend byłby **wystawiony na cały internet** — każdy mógłby uderzać w `http://<IP>:3001/api/...` z pominięciem Nginxa, certyfikatu SSL i rate-limitingu.

`DATABASE_URL: …@db:5432…` — `db` to nazwa usługi w Compose. Docker tworzy prywatną sieć, w której kontenery widzą się po nazwach jak po DNS-ie. Backend nie zna żadnego IP bazy, zna tylko nazwę „db".

`depends_on: condition: service_healthy` — Compose czeka aż healthcheck Postgresa (`pg_isready`) zwróci sukces zanim uruchomi backend. Bez tego wyścig: Node startuje w ~1s, Postgres potrzebuje ~3s → Node dostaje `ECONNREFUSED` i crashuje.

`volumes: pgdata:/var/lib/postgresql/data` — nazwany volume. Dane żyją poza kontenerem — `docker compose down` nie kasuje bazy. `docker compose down -v` kasuje. Mocno uważać na to `-v`.

7.3 `Dockerfile` backendu

```
FROM node:20-alpine
WORKDIR /app
COPY backend/package*.json ./
RUN npm install --omit=dev
COPY backend/ ./
EXPOSE 3000
CMD ["node", "src/index.js"]
```

Dlaczego `COPY package*.json` przed `COPY backend/`? Docker cache'uje warstwy. Dopóki `package.json` się nie zmienia, warstwa `npm install` (najwolniejsza) pozostaje z cache'u. Gdybyś skopiował wszystko razem, każda zmiana kodu invalidowałaby cache i musiałbyś `npm install` od nowa za każdym buildem (30+ s marnowane).

Dlaczego `alpine`? Bazowy obraz 5 MB zamiast 200 MB. Szybszy deploy, mniejsze ataki (mniej pakietów = mniej CVE). Minus: czasem trzeba doinstalować `libc6-compat` dla natywnych binariów.

Dlaczego `--omit=dev`? Pomijamy devDependencies (eslint, vitest, itp.) — produkcyjny obraz mniejszy i bezpieczniejszy.

7.4 `.env` z hasłami

```
# .env (w .gitignore!)
DB_PASSWORD=superlosowehaslo
```

Compose automatycznie podmienia `${DB_PASSWORD}` w `docker-compose.yml`.

Co jeśli nie będzie `.env`? Compose wypisze warning i wstawi puste hasło — Postgres wystartuje bez hasła. To bardzo źle.

Alternatywa produkcyjna: AWS Secrets Manager, HashiCorp Vault, Doppler. Dla MVP `.env` na serwerze wystarczy, byle nie w repo.

---

8. Proxy w Vite (dev-time)

8.1 Problem

W **dev** frontend chodzi na `http://localhost:5173` (Vite), backend na `http://localhost:3001`. Gdy w kodzie piszesz `fetch('/api/seniors')`, przeglądarka tłumaczy to na `http://localhost:5173/api/seniors` → 404.

8.2 Rozwiązanie

W `frontend/vite.config.js`:

```
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:3001',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, ''),
      },
    },
  },
});
```

Dlaczego `rewrite`? Nginx w produkcji robi `proxy_pass http://127.0.0.1:3001/` — **trailing slash obcina prefiks `/api`**. Backend zawsze widzi `GET /seniors`, nie `GET /api/seniors`. Gdyby dev nie robił tego samego, frontend pisany pod `/api/*` działałby lokalnie ale nie w produkcji (lub odwrotnie).

`★ Insight ─────────────────────────────────`

- **Parzystość dev/prod** to mantra. Wszystko co różni się między dev a prod to miejsce gdzie bug się wkradnie cicho. Proxy z rewrite to prosty sposób żeby frontend mógł pisać `/api/*` wszędzie i nie myśleć.
- Alternatywa: dodać `VITE_API_URL=http://localhost:3001` w dev i `VITE_API_URL=https://mi.zur-i.com/api` w prod. Działa, ale rozsiewasz ścieżki API po kodzie i trudniej je później zmienić.

`─────────────────────────────────────────`

8.3 Co jeśli nie zrobimy proxy?

- Frontend nie rozmawia z backendem lokalnie → nie da się testować.
- Albo: musisz pisać pełne URL-e `fetch('http://localhost:3001/seniors')` i dodać CORS po stronie backendu → dwie rzeczy do utrzymania.

---

9. React Router i pułapka odświeżania strony

9.1 Jak działa routing w SPA

React Router **symuluje** nawigację w przeglądarce:

- Klikasz `<Link to="/dashboard">` → zmienia URL przez `history.pushState` → React renderuje komponent `<Dashboard />` bez pytania serwera.
- URL wygląda jak `/dashboard` ale serwer **nigdy nie dostał requestu** o `/dashboard`.

9.2 Pułapka F5

Użytkownik jest na `/dashboard`, wciska F5 (odśwież). Teraz przeglądarka **wysyła request HTTP** `GET /dashboard` do serwera.

Problem: na serwerze nie ma pliku `/var/www/mi/dashboard`. Nginx zwraca 404.

9.3 Rozwiązanie — fallback na `index.html`

```
location / {
    try_files $uri $uri/ /index.html;
}
```

`try_files`:

1. Sprawdź czy istnieje plik o dokładnie tym URL-u.
2. Jeśli nie, sprawdź czy to folder z `index.html`.
3. Jeśli nie, **serwuj** `/index.html` (z kodem 200, nie 404).

Efekt: każdy URL który nie jest API i nie jest konkretnym plikiem → `index.html` → React startuje → patrzy na URL → renderuje `Dashboard`.

9.4 Alternatywa — hash routing

```
<HashRouter>
```

URL-e wyglądają jak `mi.zur-i.com/#/dashboard`. Wszystko po `#` przeglądarka traktuje jako fragment strony, więc F5 zawsze pyta o `/` i `index.html` jest serwowany.

Minusy: brzydkie URL-e, gorsze SEO, analytics liczy wszystkie hash-owe URL-e jako jedną stronę.

9.5 Alternatywa — Next.js z SSR

Next.js renderuje strony **po stronie serwera** — każdy URL ma odpowiadający plik lub funkcję. Żaden trick z `try_files` nie jest potrzebny, bo serwer wie co to `/dashboard`. Ale to inna architektura.

---

10. Serwer i DNS

10.1 Wynajem EC2

1. Konto na AWS.
2. EC2 → Launch Instance → Ubuntu 24.04 LTS, typ `t3.small` (1 GB RAM, ~$15/mc).

3. Security Group: otwórz porty **22 (SSH), 80 (HTTP), 443 (HTTPS)**. Nic więcej.
4. Wygeneruj parę kluczy SSH — zapisz `.pem` lokalnie.

Alternatywy tańsze/prostsze:

- **Hetzner Cloud** — VPS od €4/mc, 2x więcej RAM-u za tę cenę.
- **DigitalOcean Droplet** — $6/mc, bardzo proste UI.
- **Fly.io / Railway / Render** — zero zarządzania serwerem, deploy `git push` podobnie jak Vercel. Dla tego projektu być może lepsze niż EC2 (patrz sekcja 15).

10.2 DNS

Mamy domenę `zur-i.com`. W panelu DNS (u dostawcy gdzie kupiłeś domenę, np. Cloudflare, Namecheap, OVH):

```
Type   Name   Value
A      mi     <IP EC2>
```

Rekord A — subdomenę `mi.zur-i.com` wskazuje na IP EC2. Propagacja trwa 1-30 minut.

Co jeśli nie zrobisz DNS? Musisz wpisywać `http://54.123.45.67` zamiast `mi.zur-i.com`. Brak HTTPS (certyfikaty nie dla raw IP).

10.3 Hardening SSH

Na świeżym serwerze:

```
ssh -i klucz.pem ubuntu@<IP>

# utwórz użytkownika nie-root
sudo adduser michu
sudo usermod -aG sudo michu

# skopiuj klucz publiczny do nowego usera
sudo rsync --archive --chown=michu:michu ~/.ssh /home/michu

# wyłącz logowanie hasłem w /etc/ssh/sshd_config
# PasswordAuthentication no
# PermitRootLogin no

sudo systemctl reload ssh
```

Co jeśli nie zrobisz? Boty skanują internet 24/7 szukając otwartych SSH. Słabe hasło root = przejęcie w ciągu godzin. Klucze SSH są 1000x bezpieczniejsze.

10.4 Instalacja narzędzi na serwerze

```
# Docker
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker michu
# wyloguj i zaloguj żeby grupa zadziałała

# Node (dla buildu frontendu)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Nginx
sudo apt install -y nginx

# Certbot
sudo apt install -y certbot python3-certbot-nginx
```

Uwaga: Node na serwerze służy **tylko do buildu frontendu** przy deployu. Gdybyśmy zamiast tego budowali w CI (GitHub Actions) i wysyłali gotowe `dist/` na serwer, Node na serwerze byłby zbędny. To jest jednak optymalizacja — jak już chodzi, nie ruszaj.

---

11. Nginx

11.1 Koncepcja reverse proxy

„Proxy" — pośrednik.
„Reverse" — użytkownik nie wie że to pośrednik; myśli że rozmawia z końcowym serwerem.

Nginx stoi na porcie 443, odbiera wszystkie requesty, i decyduje:

- To plik statyczny → podaj z dysku (szybko).
- To `/api/*` → przekaż do Node (który nasłuchuje na 127.0.0.1:3001).

Czym to się różni od samego Node serwującego wszystko?

- Node jest ~5x wolniejszy w serwowaniu plików niż Nginx.
- Node nie umie natywnie HTTPS-a (umie, ale to dodatkowa komplikacja).
- Node ma jeden wątek — długi request blokuje serwer. Nginx obsługuje tysiące równolegle.
- Nginx ma wbudowany gzip, cache, rate-limit, access log.

11.2 Nasza konfiguracja (`/etc/nginx/sites-available/mi`)

```
server {
    listen 80;
    server_name mi.zur-i.com;

    root /var/www/mi;
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:3001/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

11.3 Aktywacja

```
sudo ln -s /etc/nginx/sites-available/mi /etc/nginx/sites-enabled/
sudo nginx -t              # sprawdź składnię
sudo systemctl reload nginx
```

`sudo nginx -t` to odruch życia. Zapomnienie = `systemctl reload` wywala Nginxa i strona leży.

11.4 Co robi `proxy_set_header`?

Gdy Nginx przekazuje request do Node, Node widzi Nginxa jako klienta (127.0.0.1). Traci informację kto faktycznie zapytał. Nagłówki `X-Forwarded-*` to oddają — aplikacja może je odczytać i wiedzieć prawdziwy IP, protokół (http/https).

---

12. HTTPS z Let's Encrypt

```
sudo certbot --nginx -d mi.zur-i.com
```

Certbot:

1. Sprawdza że serwer nasłuchuje na `mi.zur-i.com` (odpowiada na `.well-known/acme-challenge/...`).
2. Let's Encrypt wystawia certyfikat ważny 90 dni.

3. Certbot **modyfikuje twój** `/etc/nginx/sites-available/mi` dodając `listen 443 ssl`, ścieżki do certyfikatu, i przekierowanie `80 → 443`.
4. Dodaje do cron/systemd timer auto-odnawianie (`certbot renew`).

Co jeśli nie włączysz HTTPS?

- Przeglądarki krzyczą „Not secure".
- HTTP/2 i HTTP/3 wymagają HTTPS.
- Dane (email, hasła) lecą jawnym tekstem przez pośredników.
- Formularze z `type="password"` pokazują warning.

Alternatywa: Cloudflare przed serwerem — Cloudflare robi HTTPS, rate-limiting, DDoS protection, ty nie musisz. Minus: dane lecą przez Cloudflare.

---

13. CI/CD

13.1 Co to w ogóle jest

- **CI (Continuous Integration)** — każdy push automatycznie buduje kod i uruchamia testy. Wyłapuje regresje szybko.
- **CD (Continuous Deployment)** — każdy push na `main` automatycznie wdraża na produkcję. Brak ręcznego „teraz zrobię deploy".

Używamy **GitHub Actions** — CI/CD wbudowane w GitHub, darmowe do 2000 minut miesięcznie dla prywatnych repo.

13.2 Klucz SSH jako secret

Żeby GitHub Actions mógł się zalogować na nasz serwer:

1. Wygeneruj osobną parę kluczy **tylko do deployu**:
```
ssh-keygen -t ed25519 -C "gh-actions-deploy" -f ~/.ssh/deploy_key
```
2. Na serwerze dodaj klucz publiczny do `~/.ssh/authorized_keys` usera `michu`.
3. W GitHubie: `Settings → Secrets and variables → Actions`:
   - `EC2_HOST` = `mi.zur-i.com`
   - `EC2_USER` = `michu`
   - `EC2_SSH_KEY` = zawartość pliku `~/.ssh/deploy_key` (PRYWATNY, cały z `-----BEGIN…` do `-----END…`).

Dlaczego osobny klucz deploy'owy? Jakby ktoś włamał się do GitHuba albo wyciekł secret, rotujesz tylko ten jeden klucz, nie cały twój identyfier SSH.

13.3 Workflow (`.github/workflows/deploy.yml`)

```yaml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy over SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ${{ secrets.EC2_USER }}
          key: ${{ secrets.EC2_SSH_KEY }}
          script: |
            set -e
            cd ~/app-mi
            git fetch origin main
            git reset --hard origin/main
            git clean -fd

            docker compose up -d --build

            cd frontend
            npm install
            npm run build
            sudo rm -rf /var/www/mi/*
            sudo cp -r dist/* /var/www/mi/
            sudo chown -R www-data:www-data /var/www/mi

            docker image prune -f
```

13.4 Rozbiór

`on: push: branches: [main]` — uruchom tylko gdy zmienia się `main`. Pushe na inne branche nie deployują.

`set -e` — przerwij skrypt przy pierwszym błędzie. Bez tego błąd przy `git fetch` nie powstrzyma kolejnych poleceń — deploy może się zepsuć częściowo i zostawić serwer w dziwnym stanie.

`git fetch && git reset --hard && git clean -fd` — nie `git pull`. Dlaczego? Miałem case: server wygenerował lockfile (`npm install`) zanim my commitnęliśmy nasz. `git pull` próbował zrobić merge, dostał konflikt i się zawiesił. `reset --hard origin/main` mówi „olej lokalny stan, bądź jak GitHub". `clean -fd` kasuje untracked files żeby nie było duplikatów.

`docker compose up -d --build` — `--build` przebudowuje obraz jeśli zmienił się `Dockerfile` lub zawartość backendu. Compose jest mądry — jak nic się nie zmieniło, nie restartuje kontenera (backend żyje dalej, połączenia nie są zerwane). Jak coś zmienione, buduje, stopuje stary kontener, startuje nowy. Tutaj mamy **downtime 1-3 sekundy** przy każdym deployu.

Jak to zrobić bez downtime? Blue-green deployment — odpal nowy kontener na innym porcie, przełącz Nginx, zatrzymaj stary. Dla naszego MVP 3 sekundy to akceptowalne.

`npm install && npm run build` — buduje frontend lokalnie na serwerze. Output idzie do `frontend/dist/`.

`sudo rm -rf /var/www/mi/* && sudo cp -r dist/* /var/www/mi/` — tu jest mała **dziura atomowości**. Przez chwilę `/var/www/mi/` jest puste — jak user w tym nanosekundowym okienku odświeży stronę, dostanie 404. Profesjonalne rozwiązanie: skopiuj do `/var/www/mi-new`, potem `mv` atomowo.

`docker image prune -f` — usuwa niekuse obrazy. Inaczej po 50 deployach masz 50 starych wersji zajmujących dysk.

13.5 Idempotencja

Ten workflow można uruchomić 10 razy z rzędu — efekt będzie ten sam. Klucz:

- `reset --hard` zawsze daje ten sam stan kodu.
- `compose up` sprawdza co się zmieniło i nie robi nic niepotrzebnego.
- `rm -rf && cp` zawsze daje świeży `dist/`.

Brak idempotencji = deploy się zbije pierwszy raz jak coś wpadnie pomiędzy kroki.

13.6 Co jeśli nie mamy CI/CD?

Ręczny deploy:

```
ssh ubuntu@mi.zur-i.com
cd ~/app-mi && git pull && docker compose up -d --build && ...
```

Działa dla jednej osoby. Dla zespołu 3 osób to już:

- Zapominanie kroków.
- Różne wersje Node lokalnie u developerów (→ różne `dist/`).
- Ktoś deployuje swoje niecommitnięte zmiany.
- Brak historii „kto deployował co, kiedy".

CI/CD = deploy jest zawsze reprodukowalny ze stanu repo.

---

14. Utrzymanie

14.1 Logi

```
# backend
docker compose logs -f backend

# Nginx
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log

# system (restarty kontenerów)
journalctl -u docker
```

14.2 Backup bazy

```
# co noc przez cron
docker compose exec -T db pg_dump -U appuser appdb | gzip > /backup/$(date +%F).sql.gz
```

Upload na S3/R2 żeby nie tracił danych przy padzie dysku.

Co jeśli nie ma backupu? Padnięcie EBS = utrata wszystkiego. AWS nie ma litości.

14.3 Monitoring

Minimum: **UptimeRobot** (darmowy) — pinguje `https://mi.zur-i.com/api/health` co 5 min, wysyła email jak padnie.

Pro: Grafana + Prometheus + `node_exporter` — wykresy CPU, RAM, ruchu.

14.4 Aktualizacje systemu

```
sudo apt update && sudo apt upgrade -y
# raz w tygodniu
```

Stare Ubuntu = niezałatane CVE = ryzyko.

---

15. Alternatywy globalne

Zamiast EC2 + ręcznego stacka:

15.1 Vercel

- Frontend: `git push` → deploy 30s, CDN globalny. Darmowy do ~100GB ruchu.

- Backend: Serverless Functions (Node, max 10s wykonania). Baza: osobno.
- **Nie pasuje** gdy masz długie requesty, WebSockety, heavy background jobs.

15.2 Railway / Render

- Jeden dashboard dla frontendu, backendu i Postgresa.
- `git push` = deploy. Nie musisz dotykać SSH/Nginx/Certbota.
- Koszt: ~$5-20/mc dla tej skali.
- **Dobry wybór** gdy chcesz skupić się na kodzie, nie infrastrukturze.

15.3 Fly.io

- Docker-based, deploy przez CLI. Globalny edge.
- Postgres managed albo własny w kontenerze.
- Średni próg wejścia, ale bardzo elastyczny.

15.4 Kiedy EC2 ma sens?

- Już znasz Linuxa.
- Chcesz pełnej kontroli (specyficzne wersje, niestandardowe narzędzia).
- Masz powody żeby być w AWS (reszta infry, VPC, IAM).
- Ćwiczenie edukacyjne — rozumiesz co się pod spodem dzieje.

---

16. Checklist „od zera do produkcji"

- [ ] Lokalnie: Node 20, Docker, Git.
- [ ] `git init`, `gh repo create`.
- [ ] Frontend: `npm create vite@latest`, zainstaluj react-router-dom.
- [ ] Backend: `npm init`, zainstaluj express, pg, cors.
- [ ] `schema.sql` + `seeds/*.json`.
- [ ] `docker-compose.yml` z backendem + db.
- [ ] `.env` z `DB_PASSWORD`, dodaj do `.gitignore`.
- [ ] `vite.config.js` z proxy `/api → localhost:3001`.
- [ ] Lokalny test: `docker compose up`, `npm run dev`, klikaj.
- [ ] Kup/skonfiguruj domenę, dodaj rekord A.
- [ ] Wynajmij EC2 (albo VPS), otwórz porty 22/80/443.
- [ ] Utwórz non-root usera, wyłącz password auth SSH.
- [ ] Zainstaluj Docker + Node + Nginx + Certbot na serwerze.
- [ ] Sklonuj repo do `~/app-mi`, wrzuć `.env`.
- [ ] Napisz `nginx.conf`, symlinkuj do `sites-enabled`.
- [ ] `sudo nginx -t && systemctl reload nginx`.
- [ ] `sudo certbot --nginx -d mi.zur-i.com`.

- [ ] Wygeneruj klucz deploy'owy SSH, dodaj do GitHub Secrets.
- [ ] `.github/workflows/deploy.yml`.
- [ ] `git push origin main` — pierwszy automatyczny deploy.
- [ ] Skonfiguruj UptimeRobot na `/api/health`.
- [ ] Skonfiguruj cron dla `pg_dump` backupu.

---

17. Najczęstsze pułapki

Problem	Przyczyna	Fix
Frontend 404 po F5 na głębokim URL	brak `try_files … /index.html`	popraw Nginx
`ECONNREFUSED` przy starcie backendu	wyścig z Postgresem	`depends_on: condition: service_healthy`
Backend widziany z internetu	brak `127.0.0.1:` w porcie Compose	dodaj prefiks
Deploy rzuca „working tree would be overwritten"	`git pull` zamiast `reset --hard`	zmień workflow
Mixed content warning	HTTPS z assetami po HTTP	wszystko przez `//` lub `https://`
Certyfikat wygasł	cron certbota nie działa	`sudo systemctl status certbot.timer`
`npm install` crashuje na serwerze	Node out of memory na 1 GB RAM	build w CI, kopiuj tylko `dist/`

---

18. Źródła do pogłębienia

- **Docker** — docs.docker.com/compose.
- **Nginx** — nginx.org/en/docs/beginners_guide.html.
- **React Router** — reactrouter.com.
- **Vite** — vitejs.dev.
- **GitHub Actions** — docs.github.com/actions.
- **Let's Encrypt** — certbot.eff.org.
- **Postgres** — postgresql.org/docs/16.

Ten stack (Vite + React + Express + Postgres + Docker Compose + Nginx + GH Actions + Let's Encrypt + EC2) jest **nudny w najlepszym znaczeniu tego**

słowa — każdy kawałek jest sprawdzony w boju, ma dużą społeczność, i nic z tego nie wyjdzie z użycia w najbliższej dekadzie. To dobry fundament.